# Investment Advisor

Agentic AI system — implementation-focused technical architecture

- Python • LangGraph • LangChain • FastAPI • PostgreSQL • pgvector • Docker
- Specialized agents for query understanding, customer context, knowledge retrieval, risk, portfolio analysis and compliance
- Dynamic Routing and guardrails
- Scope: local development: Fast API
- Evaluation (LLM & LLM+RAG)
- Centralized Logs & Traces

# Project objective

UK-focused investment education and controlled guidance

- Personalized Investment Guidance: Combine customer financial context with knowledge base
- Intelligent Multi-Agent Automation: Implement a dynamic LangGraph workflow with specialized agents, parallel execution, and RAG-based knowledge retrieval to improve efficiency, accuracy, and explainability.
- Secure & Compliant AI: Ensure data privacy, regulatory compliance, and responsible investment guidance

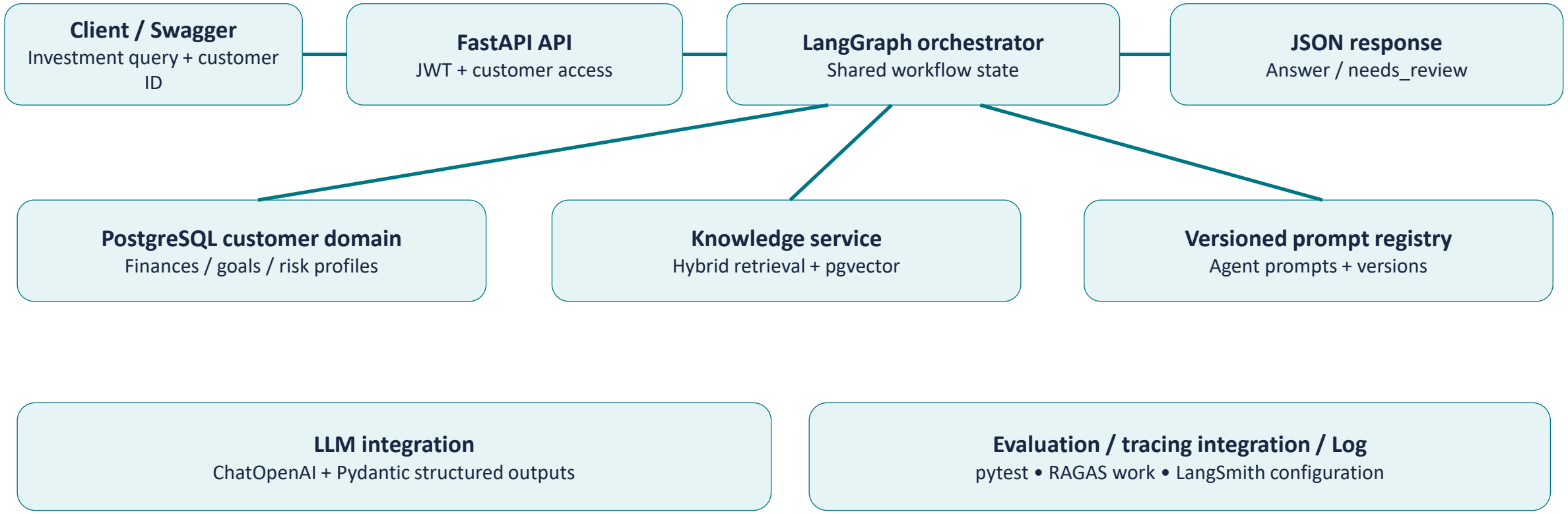
# Application architecture

Client → secured API → LangGraph → PostgreSQL and RAG

- FastAPI exposes login, current-user, health and investment-analysis routes
- JWT authentication and per-customer authorization precede agent execution
- LangGraph coordinates the query, customer, knowledge, risk, portfolio and compliance steps
- PostgreSQL stores customer data, authentication mappings, knowledge and prompt versions

# Application architecture — end-to-end

Request path, agent orchestration, data services and response handling

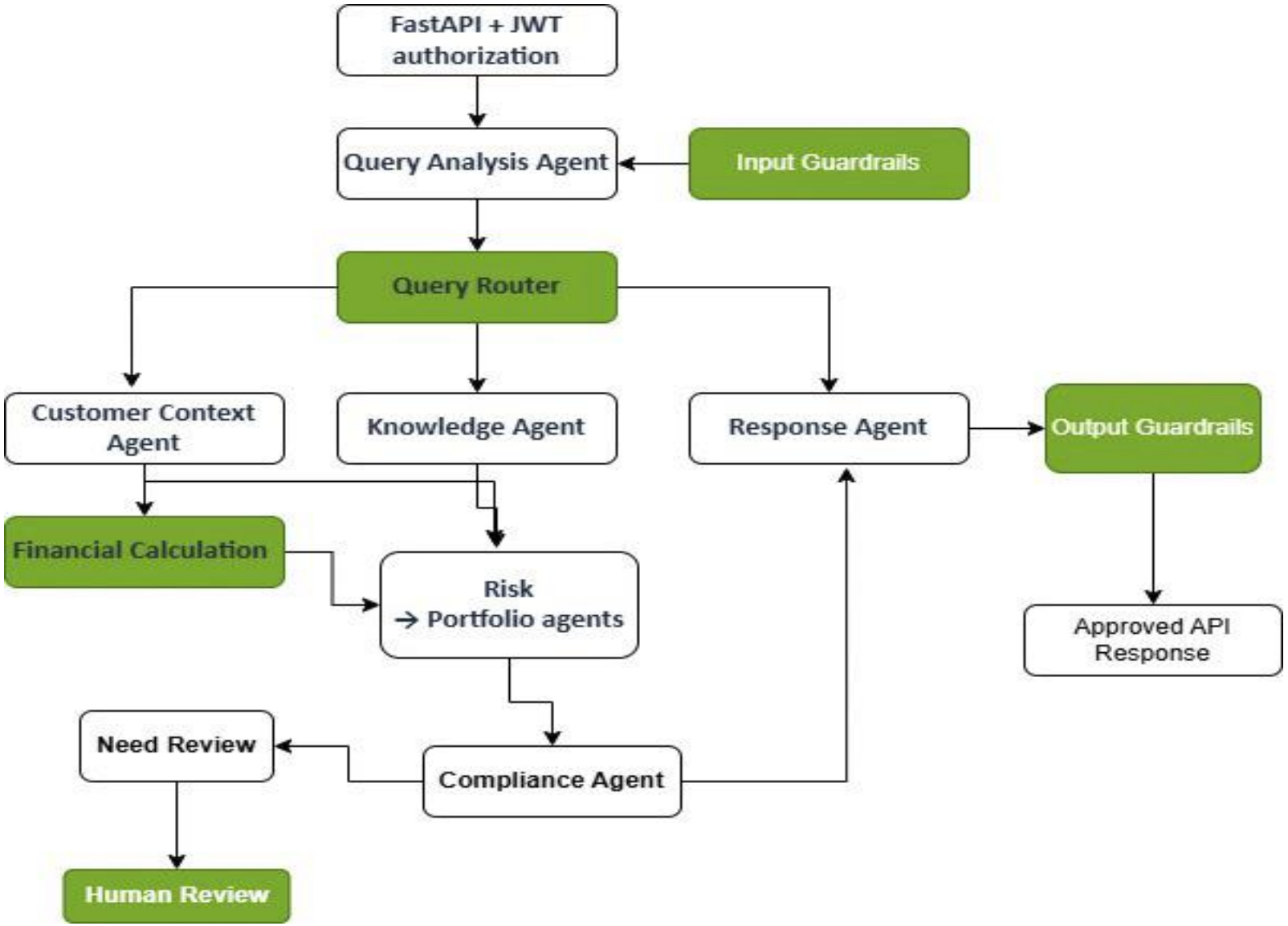


# LangGraph workflow

- Query Understanding: Query Agent uses an LLM to classify user intent, identify requirements, and detect missing information.
- Dynamic Planning & Routing: A rule-based Planner selects required agents based on intent, supports conditional execution, and avoids unnecessary LLM calls.
- Customer Data & Knowledge Retrieval
- Risk & Portfolio Analysis: Risk Agent evaluates financial capacity and risk tolerance; Portfolio Agent generates personalized allocation proposals based on customer goals and constraints.
- Compliance & Guardrails: Compliance Agent validates recommendations against investment policies, suitability rules, and risk limits, escalating cases requiring human review.
- Response Generation & Evaluation: Response Agent generates grounded, explainable answers.

# Agentic AI — LangGraph workflow

- Implemented graph structure



# RAG implementation

Development knowledge base and retrieval pipeline

- Document Ingestion: Load PDF, TXT, and Markdown files, perform chunking, generate OpenAI embeddings, and store them in PostgreSQL with pgvector.

- Hybrid Retrieval: Combine BM25 keyword search and vector similarity search to retrieve relevant document chunks.

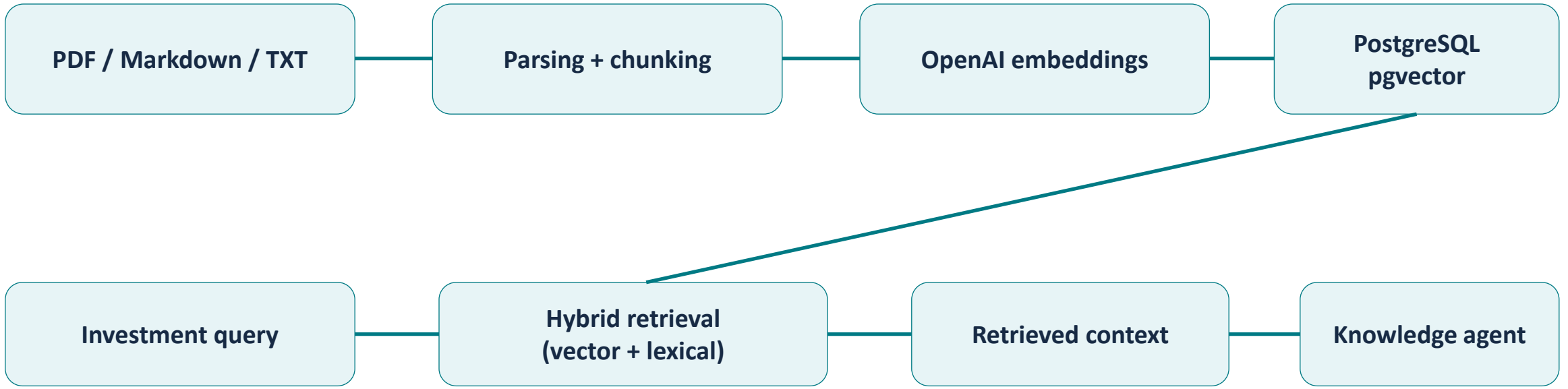
- Reranking: select the most relevant Top-N chunks

- Reranking: Incremental Ingestion: Process only new or modified documents to avoid duplicate embeddings and reduce costs.

- Cleanup & Deletion: Remove deleted documents and outdated chunks from PostgreSQL to keep the knowledge base synchronized.

# RAG — ingestion and retrieval architecture

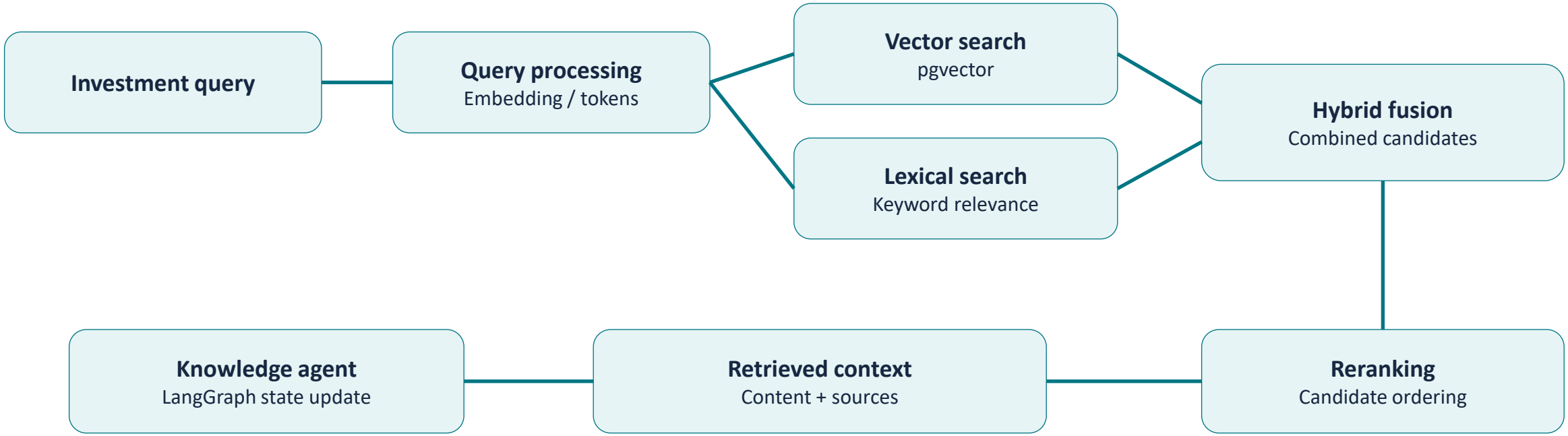
Development knowledge base: sample investment education, risk and compliance documents




Scope: ingestion and hybrid-retrieval components developed locally. Source policies are illustrative; retrieval quality and reranking have not been independently benchmarked.

# RAG architecture — online retrieval

Semantic + lexical retrieval, context assembly and agent handoff



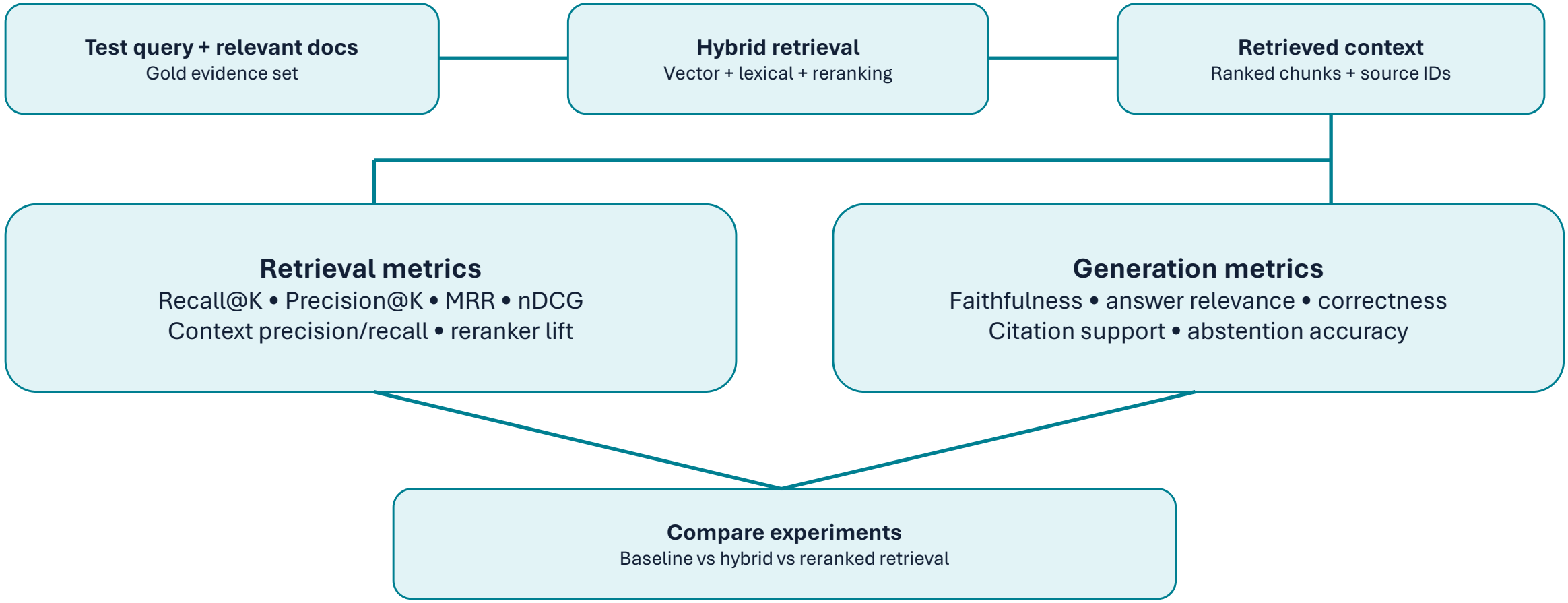

Boundary: retrieval and reranking components were developed, but retrieval accuracy, reranker lift and end-to-end benchmarks have not been independently established.

# Evaluation Metrics

- **Retrieval Evaluation:**
  - Precision@K: Measures how many of the top-K retrieved documents are relevant.
  - Recall@K: Measures how many relevant documents are retrieved within the top-K results.
  - MRR (Mean Reciprocal Rank): Measures how early the first relevant document appears in search results.
  - NDCG: Measures ranking quality by rewarding relevant documents appearing higher in the results.
  - Latency
  - Hit Rate@K: Measures whether at least one relevant document appears in the top K results.
- **Generation Evaluation (RAGAS):**
  - Faithfulness: Measures whether the generated answer is supported by the retrieved context.
  - Response Relevancy: Measures how well the generated answer addresses the user's question.

# RAG + LLM evaluation

Measure retrieval independently before scoring grounded generation



# Guardrails: Input & Output Guardrails, and PII Masking

- Input & Output Validation: Reject empty queries and enforce a 4,000-character limit before LangGraph execution.
- Prompt Injection Protection: Detect and block known attempts to override system instructions, disable safety controls, or bypass compliance.
- PII Detection & Masking: Use spaCy NER and regex to identify and mask names, emails, phone numbers, DOB, PAN, Aadhaar, and National Insurance numbers.
- Sensitive Data Leakage Prevention: Screen generated responses for sensitive identifiers and block responses containing detected PII.

# FastAPI and JWT

Local API implementation and security boundary

- GET /health; POST /api/v1/auth/login; GET /api/v1/auth/me
- POST /api/v1/investment/analyze invokes the graph after customer access validation
- Argon2 password hashing (pwdlib), PyJWT and async SQLAlchemy

# Data architecture

- Customer tables: employment, finances, liabilities, goals, investments, preferences and risk profiles
- api_users stores hashed credentials; api_user_customers maps permitted customer access
- agent_prompts stores agent-specific system prompts and active versions

## Future Improvements (if required) and Production Roadmap

1. **Security / Guardrail**: RBAC, prompt injection protection, rate limiting and token management.
2. **Agent & Tool Security**: Tool permissions, input validation, execution limits and access control.
3. **RAG Enhancement**: Document versioning, source verification, access control and retrieval optimization.
4. **Financial Compliance:** Suitability engine, FCA policy enforcement, risk assessment and explainable decisions.
5. **Human-in-the-Loop:** Approval workflows, adviser dashboard and LangGraph checkpointing.
6. **Reliability & Performance:** Retry mechanisms, caching, error handling, latency and token-cost optimization.
7. **Testing & Evaluation:** security testing, regression testing and load testing.
8. **Monitoring & Auditing:** agent tracing, audit trails and automated alerts.
9. **GCP Deployment:** Docker, Cloud Run, Cloud SQL, Secret Manager and Redis.
10. **Scalability:** Auto-scaling, asynchronous processing, database optimization and disaster recovery.

# Next steps: GCP deployment + CI/CD

Future work only — not implemented

- Target Google Cloud Platform (GCP): Cloud Run for FastAPI + LangGraph
- Stabilize local workflow, authentication and regression tests; pin dependencies
- Build Docker image; publish to Artifact Registry; use Secret Manager and Cloud SQL PostgreSQL
- GitHub Actions CI: lint, pytest, dependency checks, Docker build; Workload Identity Federation
- GitHub Actions CD: deploy to Cloud Run, approvals, rollback and monitoring